

3. СИСТЕМА КОМАНД ЭВМ СЕМЕЙСТВА VAX11

ЭВМ семейства VAX обладают развитой системой команд, которая включает более 300 операций, реализуемых аппаратно. В эмулирующей программе реализовано 87 операций.

Команды ЭВМ семейства VAX – переменной длины, с выравниванием по границе байта. Такой формат делает команды компактными и позволяет расширять их набор. Код операции может занимать один или два байта. В зависимости от команды за кодом операции следует от 0 до 6 спецификаций операндов.

Команды, реализованные в эмулирующей программе, можно разделить на следующие основные группы:

- выполнения целочисленных арифметических и логических операций,
- пересылки,
- безусловного переходов и обращения к подпрограммам,
- условных переходов по кодам условий,
- организации циклов,
- специального назначения.

В данном пособии не рассматриваются команды, не вошедшие в состав эмулирующей программы (выполнение операций над числами с плавающей запятой и символьными строками, команды деления и умножения для целых чисел, команды работы со списками, указателями и т. д.).

Для удобства изложения информация по каждой группе команд сведена в отдельные таблицы. Кроме того, в Приложении 2 приводится общий список команд, реализованных в эмулирующей программе. В описаниях команд операнды имеют обозначение «опN», где N – целое число от 1 до 6. Коды операций приводятся в шестнадцатеричной системе счисления. В мнемоническом обозначении операции после названия могут быть указаны: тип данных, с которым работает данная команда (B, W, L) и число используемых операндов (2 или 3). По умолчанию, если в команде не указан тип данных, то команды выполняются в формате L, например команды ADWC и SBWC. Для тех команд, которые формируют новые значения признаков (N, Z, V, C), указывается, какие именно признаки устанавливаются по результатам выполнения этих команд. При этом используются следующие обозначения:

«+» – значение признака устанавливается по результатам выполнения данной команды;

«-» – значение признака в результате выполнения данной команды не изменяется, т. е. сохраняется предыдущее значение.

3.1. Команды выполнения арифметических и логических операций над целыми числами

Команды, относящиеся к этой группе, приведены в табл. 3.1.

Таблица 3.1

Целочисленные арифметические и логические операции

Мнемо-код	КОП	Операция	Флаги				Описание
			N	Z	V	C	
ADDB2	80	оп2 = оп2 + оп1 (двухоперандное сложение)	+	+	+	+	оп1 и оп2 складываются, результат записывается по адресу оп2
ADDW2	A0		+	+	+	+	
ADDL2	C0		+	+	+	+	
ADDB3	81	оп3 = оп2 + оп1 (трехоперандное сложение)	+	+	+	+	оп1 и оп2 складываются, результат записывается по адресу оп3
ADDW3	A1		+	+	+	+	
ADDL3	C1		+	+	+	+	
SUBB2	82	оп2 = оп2 – оп1 (двухоперандное вычитание)	+	+	+	+	оп1 вычитается из оп2, результат записывается по адресу оп2
SUBW2	A2		+	+	+	+	
SUBL2	C2		+	+	+	+	
SUBB3	83	оп3 = оп2 – оп1 (трехоперандное вычитание)	+	+	+	+	оп1 вычитается из оп2, результат записывается по адресу оп3
SUBW3	A3		+	+	+	+	
SUBL3	C3		+	+	+	+	
ADWC	D8	оп2 = оп2 + оп1 + C (сложение с переносом)	+	+	+	+	оп1 складывается с оп2 и значением признака C, результат записывается по адресу оп2
SBWC	D9	оп2 = оп2 – оп1 – C (вычитание с заемом)	+	+	+	+	оп1 и значение признака C вычитаются из оп2, результат записывается по адресу оп2
INCB	96	оп1 = оп1 + 1 (инкремент)	+	+	+	+	оп1 складываются с 1, результат записывается по адресу оп1
INCW	B6		+	+	+	+	
INCL	D6		+	+	+	+	
DECB	97	оп1 = оп1 – 1 (декремент)	+	+	+	+	Из оп1 вычитается 1, результат записывается по адресу оп1
DECW	B7		+	+	+	+	
DECL	D7		+	+	+	+	
MNEGB	8E	оп2 = – оп1 (пересылка дополнения оп1)	+	+	+	+	Дополнение оп1 пересылается по адресу оп2
MNEGW	AE		+	+	+	+	
MNEGL	CE		+	+	+	+	
MCOMB	92	оп2 = оп1 (пересылка инверсии)	+	+	0	–	Инверсия оп1 пересылается по адресу оп2
MCOMW	B2		+	+	0	–	
MCOML	D2		+	+	0	–	

Продолжение табл. 3.1

Мнемо-код	КОП	Операция	Флаги				Описание
			N	Z	V	C	
CLRB	94	оп1 = 0 (очистка операнда)	0	1	0	–	По адресу оп1 заносится 0
CLRW	B4		0	1	0	–	
CLRL	D4		0	1	0	–	
CLRQ	7C		0	1	0	–	
CLRO	7CFD		0	1	0	–	
BISB2	88	оп2 = оп2 v оп1 (двухоперандная установка разрядов)	+	+	0	–	Логическое сложение оп1 и оп2 , результат записывается по адресу оп2
BISW2	A8		+	+	0	–	
BISL2	C8		+	+	0	–	
BISB3	89	оп3 = оп2 v оп1 (трехоперандная установка разрядов)	+	+	0	–	Логическое сложение оп1 и оп2 , результат записывается по адресу оп3
BISW3	A9		+	+	0	–	
BISL3	C9		+	+	0	–	
BICB2	8A	оп2 = оп2 & $\overline{\text{оп1}}$ (двухоперандная очистка разрядов)	+	+	0	–	оп2 логически умножается на инверсию оп1 , результат записывается по адресу оп2
BICW2	AA		+	+	0	–	
BICL2	CA		+	+	0	–	
BICB3	8B	оп3 = оп2 & $\overline{\text{оп1}}$ (трехоперандная очистка разрядов)	+	+	0	–	оп2 логически умножается на инверсию оп1 , результат записывается по адресу оп3
BICW3	AB		+	+	0	–	
BICL3	CB		+	+	0	–	
XORB2	8C	оп2 = оп2 + оп1 (двухоперандное исключающее ИЛИ)	+	+	0	–	Исключающее ИЛИ между оп1 и оп2, результат запоминается по адресу оп2
XORW2	AC		+	+	0	–	
XORL2	CC		+	+	0	–	
XORB3	8D	оп3 = оп2 + оп1 (трехоперандное исключающее ИЛИ)	+	+	0	–	Исключающее ИЛИ между оп1 и оп2, результат запоминается по адресу оп3
XORW3	AD		+	+	0	–	
XORL3	CD		+	+	0	–	
CMPB	91	оп1 – оп2 (сравнение)	+	+	0	+	Из оп1 вычитается оп2, результат не запоминается, используется для установки признаков
CMPW	B1		+	+	0	+	
CMPL	D1		+	+	0	+	
TSTB	95	оп1 = оп1 (опрос слова – тестирование)	+	+	0	0	В соответствии с оп1 устанавливаются признаки
TSTW	B5		+	+	0	0	
TSTL	D5		+	+	0	0	
BITB	93	оп2 & оп1 (проверка разрядов)	+	+	0	–	оп1 логически умножается на оп2, результат не запоминается, используется для установки признаков
BITW	B3		+	+	0	–	
BITL	D3		+	+	0	–	

Мнемо-код	КОП	Операция	Флаги				Описание
			N	Z	V	C	
ASHL	78	$оп3 = оп2 * 2^{оп1}$	+	+	+	0	оп2 сдвигается на число разрядов, указанное в оп1, результат запоминается по адресу оп3. Если $оп1 > 0$ – сдвиг влево, иначе ($оп1 < 0$) – сдвиг вправо
ASHQ	79	(арифметический сдвиг: L – формат длинного слова, Q – формат квадрослова)	+	+	+	0	

Большинство команд, приведенных в табл. 3.1, довольно просты и не требуют дополнительных пояснений. Исключение составляют лишь команды сдвига и сложения (вычитания) с переносом, которые мы рассмотрим ниже. Следует также отметить, что код операции CLRO занимает 2 байта (значение младшего байта – FD, старшего байта – 7C).

3.1.1. Команды сдвига

Команды ASHL и ASHQ выполняют арифметический сдвиг, в результате которого знак числа сохраняет свое положение. Команда ASHL используется для сдвига длинных слов, а команда ASHQ – для сдвига квадрослов. Команды имеют следующий формат:

ASHL оп1,оп2,оп3,
ASHQ оп1,оп2,оп3.

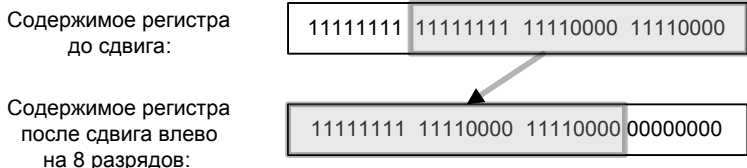
Первый операнд (оп1) – 8-разрядное число со знаком, определяющее количество разрядов, на которое следует произвести сдвиг и направление сдвига. Если $оп1 > 0$, то производится сдвиг влево. Если $оп1 < 0$, то происходит сдвиг вправо. Фактически это аналогично умножению на 2 в степени оп1 или делению на 2 в степени оп1. Если $оп1 = 0$ – сдвиг не выполняется.

При выполнении команд ASHL и ASHQ содержимое оп2 сдвигается на число разрядов, указанных в оп1, и результат помещается по адресу оп3. Содержимое оп2 в результате выполнения команды сдвига не изменяется. Команды сдвига устанавливают биты N и Z в зависимости от значения результата. Бит V будет установлен, если при сдвиге влево получен результат, значение которого выходит за пределы диапазона представимых чисел (т. е. старшая значащая единица выходит за пределы разрядной сетки, что приводит к изменению знака числа). Так как эти инструкции предна-

значены для работы только с числами со знаком, бит *C* всегда сбрасывается.

Пример 1. Необходимо выполнить команду `ASHL #8,R0,R0` (сдвиг *R0* влево на 8 разрядов). Процесс сдвига показан рис. 3.1, а выполнение команды на эмуляторе на рис. 3.2 и 3.3.

Пример 2. Необходимо выполнить команду `ASHL #-8,R0,R0` (сдвиг *R0* влево на 8 разрядов). Процесс сдвига показан рис. 3.4, а выполнение команды на симуляторе – на рис. 3.5 и 3.6.



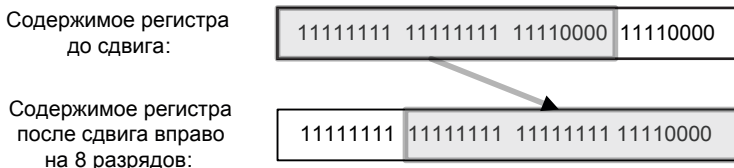
*Рис. 3.1. Арифметический сдвиг *R0* влево на 8 разрядов*

Редактор памяти																Регистры и флаги	
	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F	Флаги
00000000	78	08	50	50	00	00	00	00	00	00	00	00	00	00	00	00	T=0 N=0
00000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Рис. 3.2. Команда арифметического сдвига влево

Редактор памяти																Регистры и флаги	
	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F	Флаги
00000000	78	08	50	50	00	00	00	00	00	00	00	00	00	00	00	00	T=0 N=1
00000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Рис. 3.3. Результат выполнения команды арифметического сдвига влево



*Рис. 3.4. Арифметический сдвиг *R0* вправо на 8 разрядов*

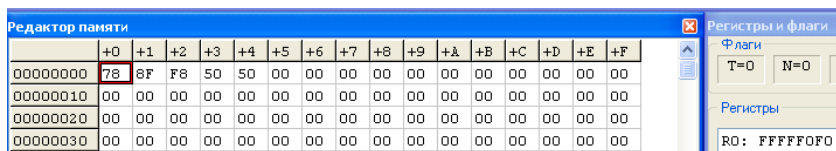


Рис. 3.5. Команда арифметического сдвига вправо

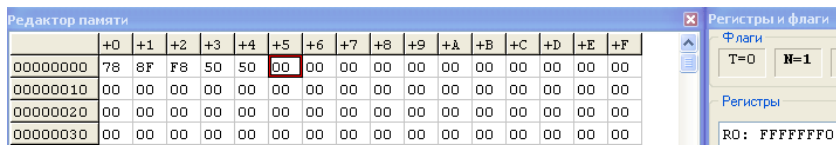


Рис. 3.6. Результат выполнения команды арифметического сдвига вправо

3.1.2. Команды сложения и вычитания с переносом

В ЭВМ семейства VAX бит C показывает наличие переноса при сложении или заема при вычитании младших частей больших чисел. Следовательно, для учета переноса (заема) необходимо прибавить (вычесть) значение бита C к (из) старшей части результата. Конечно, можно было бы проверить значение бита C, а затем увеличить (или уменьшить) результат на 1. Чтобы не выполнять последовательно эти действия, разработчики ЭВМ VAX ввели следующие команды: ADWC – сложить с переносом, SBWC – вычесть с заемом. Обе команды работают с операндами, имеющими формат длинных слов. Эти команды могут быть использованы для выполнения вычислений с двойной точностью, например с 64-битовыми словами, совместно с другими арифметическими операциями.

П р и м е р 3. Пусть два 64-разрядных числа A и B содержатся в длинных словах с адресами AH, AL, BH, BL, где буквы H и L обозначают старшую и младшую часть числа соответственно. Сложение с двойной точностью осуществляется следующей последовательностью команд:

ADDL2 AL, BL
ADWC AH, BH.

Пусть $A = -1$ и $B = -1$. Разместим число A в регистрах R0(AL) и R1(AH), а число B в R2(BL) и R3(BH). Выполнение операции сложения представлено на рис. 3.7, результаты выполнения сложения на симуляторе – на рис. 3.8–3.10.

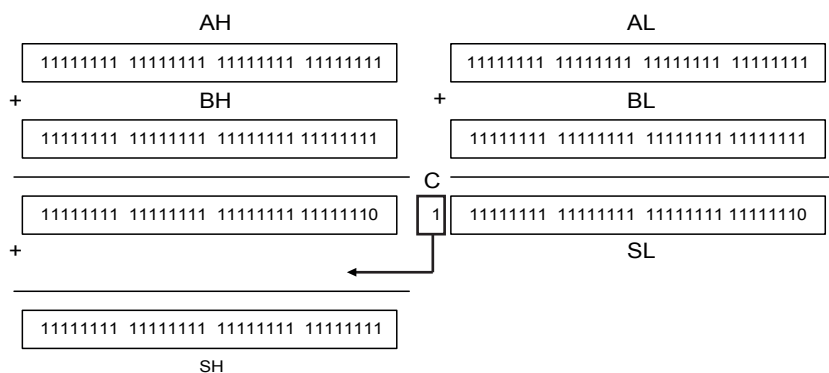


Рис. 3.7. Сложение чисел, представленных в формате квадрослова

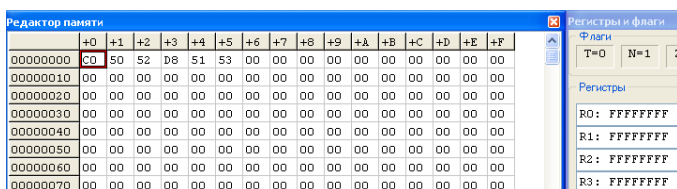


Рис. 3.8. Программа сложения двух чисел в формате квадрослова

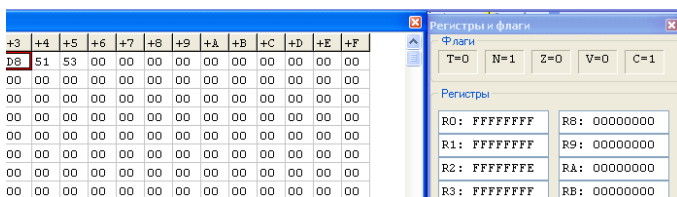


Рис. 3.9. Результат сложения младших частей чисел

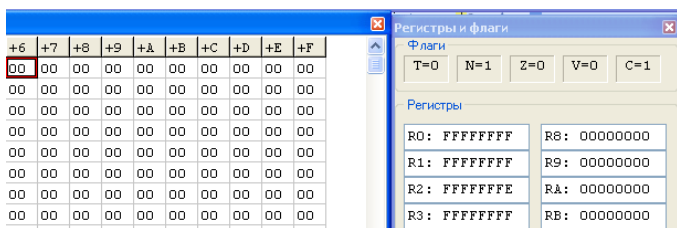


Рис. 3.10. Результат сложения старших чисел и переноса от предыдущей операции сложения

Аналогично вычитание A из B производится командами:

SUBL2 AL, BL

SBWC AH BH.

3.2. Команды пересылки

Команды пересылки приведены в табл. 3.2.

Код операции MOVO (переслать октаслово) состоит из двух байтов (значение младшего байта – FD, старшего байта – 7D).

Команды пересылки формируют флаги N и Z, т. е. пересылка выполняется через АЛУ, как некоторая арифметико-логическая операция между первым операндом и константой «0».

Таблица 3.2

Операции пересылки

Мнемокод	КОП	Операция	Флаги				Описание
			N	Z	V	C	
MOVB	90	оп2 = оп1 (пересылка операндов различного формата)	+	+	0	–	оп1 пересылается по адресу оп2
MOVW	B0		+	+	0	–	
MOVL	D0		+	+	0	–	
MOVQ	7D		+	+	0	–	
MOVO	7DFD		+	+	0	–	

3.3. Команды безусловного перехода и обращения к подпрограммам

Описание команд этой группы приведено в табл. 3.3. При их выполнении содержимое регистра слова состояния процессора PSW не изменяется.

Логика выполнения команд BRB и BRW рассмотрена в п. 2.9. Данные команды имеют ограничения, связанные с тем, что смещение, указываемое в этих командах, позволяет осуществлять переход лишь на определенное число байтов (в зависимости от разрядности смещения). Например, команда BRW может осуществлять переход на 32 765 байтов назад или на 32 770 байтов вперед от адреса команды BRW. Если необходимо осуществить переход на большее число байтов, то использовать данную команду нельзя.

Команды безусловного перехода и обращения к подпрограммам

Мнемокод	КОП	Название и формат	Описание
BRB	11	Безусловный переход (смещение в формате байта) BRB-смещение	Переход по адресу, получаемому путем прибавления к R15 смещения в формате байта или слова
BRW	31	Безусловный переход (смещение в формате слова) BRW-смещение	
JMP	17	Универсальный переход JMP оп1	В R15 заносится адрес оп1
JSB	16	Переход к подпрограмме JSB оп1	R15 запоминается в стеке, затем в R15 заносится адрес оп1
RSB	05	Возврат из подпрограммы RSB	В R15 заносится длинное слово, выбранное из стека

Команда JMP не имеет таких ограничений, как рассмотренные выше команды, и позволяет осуществлять переход по всему адресному пространству. В отличие от других команд перехода она имеет формат, согласующийся с форматами остальных команд ЭВМ VAX. При этом в счетчик команд PC заносится адрес, определяемый в соответствии со способом адресации, указанным в спецификации операнда.

П р и м е р. Пусть необходимо выполнить переход с адреса ^X 00000000 на адрес ^X 00008026. Для передачи управления по данному адресу нельзя использовать команды BRB и BRW, так как величина смещения больше, чем 32 770. Поэтому используют команду JMP с относительной адресацией со смещением в формате длинного слова (рис. 3.11). Для демонстрации этой команды в эмуляторе необходимо расширить адресное пространство ОЗУ, которое по умолчанию составляет 2048 байт (см. Приложение 2). К тому моменту, когда из памяти производится выборка смещения ^X 00008020, содержимое PC увеличивается до ^X 00000006. Таким образом, по этой команде будет выполняться переход по адресу ^X 00000006 + ^X 00008020, т. е. по адресу ^X 00008026 (рис. 3.12). Дополнительное преимущество команды JMP состоит в том, что она может применяться с большим числом способов адресации, что позволяет легко организовать сложные управляющие конструкции.

Редактор памяти																
	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
00000000	17	EF	20	80	00	00	00	00	00	00	00	00	00	00	00	00
00000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

RF: 00000000

Рис. 3.11. Начальное содержимое редактора памяти

Редактор памяти																
	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
00008010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00008020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

RF: 00008026

Рис. 3.12. Содержимое редактора памяти после выполнения команды перехода

Команда перехода к подпрограмме JSB по формату аналогична команде JMP. Для хранения адреса возврата в основную программу при обращении к подпрограмме служит специально отведенная область ОЗУ, называемая *стеком*. Для обращения к стеку используется регистр 14 (SP) – указатель стека. При использовании этого регистра в других целях необходимо быть внимательным, так как это может привести к нарушению правильных связей в программе при очередном вызове подпрограммы или возврате из нее. По команде JSB значение указателя стека уменьшается на 4, в стеке сохраняется значение программного счетчика, после чего в PC заносится адрес, сформированный в соответствии с указанным способом адресации. Выполнение команды JSB op1 эквивалентно выполнению следующих команд:

MOVL PC, -(SP)
JMP op1

П р и м е р. Пусть необходимо выполнить безусловный переход с адреса ^X 00000040 на подпрограмму, которая начинается с адреса ^X 00000100. Отведем область памяти под стек, начиная с адреса ^X 00000150 (стек растет в сторону уменьшения адресов). Этот адрес занесем в регистр 14. В команде JSB используем абсолютную адресацию. Содержимое редактора памяти до и после выполнения команды перехода на подпрограмму представлено на рис. 3.13 и 3.14 соответственно.

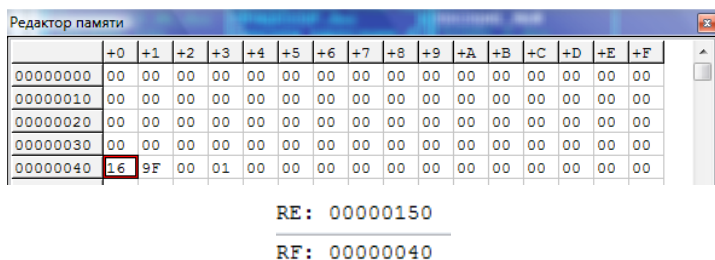


Рис. 3.13. Содержимое редактора памяти до выполнения команды JSB

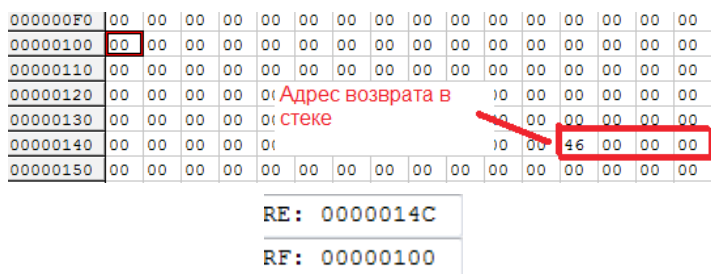


Рис. 3.14. Содержимое редактора памяти после выполнения команды JSB

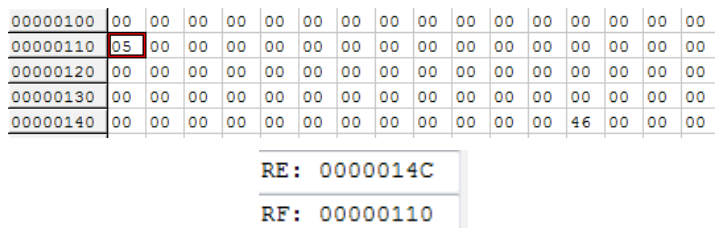


Рис. 3.15. Содержимое редактора памяти до выполнения команды RSB

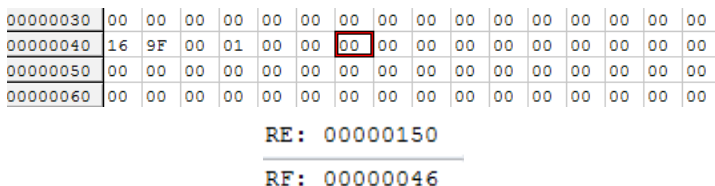


Рис. 3.16. Содержимое редактора памяти после выполнения команды RSB

Команда возврата из подпрограммы RSB не имеет операндов. По этой команде считывается адрес возврата из стека по адресу, указанному в регистре R14, и записывается в PC, после чего указатель стека увеличивается на 4. Команда RSB эквивалентна команде MOVL (SP)+, PC. Пусть в подпрограмме команда возврата находится по адресу ^X 00000110. Содержимое редактора памяти до и после выполнения команды возврата из подпрограммы представлено на рис. 3.15 и 3.16 соответственно.

3.4. Команды перехода по кодам условий

Для наглядности изложения команды перехода по кодам условий (табл. 3.4) выделены отдельно от других команд перехода. При выполнении этих команд содержимое регистра PSW не изменяется.

Команды условного перехода BGEQ, BGTR, BLSS и BLEQ используются после сравнения чисел со знаком. Их не следует применять при работе с числами без знака. Например, если рассматривать содержимое длинных слов как число без знака, то число ^X FFFFFFFF больше числа ^X 00000000. Однако операция сравнения, предшествующая команде BGTR, даст противоположный результат, поскольку число ^X FFFFFFFF будет рассматриваться как отрицательное, а именно как -1 . Для решения этой проблемы предусмотрены команды BGEQU, BGTRU, BLSSU и BLEQU, применяемые для перехода после сравнения чисел без знака. Эти команды имеют смысл только тогда, когда они используются после операций сравнения.

Таблица 3.4

Команды перехода по кодам условий

Мнемокод	КОП	Название	Значение кодов условий в PSW для перехода
BNEQ	12	Переход, если не равно (со знаком)	$Z = 0$
BNEQU	12	Переход, если не равно (без знака)	$Z = 0$
BEQL	13	Переход, если равно (со знаком)	$Z = 1$
BEQLU	13	Переход, если равно (без знака)	$Z = 1$
BGTR	14	Переход, если больше (со знаком)	$Z \vee (N + V) = 0$
BLEQ	15	Переход, если меньше или равно (со знаком)	$Z \vee (N + V) = 1$

Мнемокод	КОП	Название	Значение кодов условий в PSW для перехода
BGEQ	18	Переход, если больше или равно (со знаком)	$(N + V) = 0$
BLSS	19	Переход, если меньше (со знаком)	$(N + V) = 1$
BGTRU	1A	Переход, если больше (без знака)	$C \vee Z = 0$
BLEQU	1B	Переход, если меньше или равно (без знака)	$C \vee Z = 1$
BVC	1C	Переход, если нет переполнения	$V = 0$
BVS	1D	Переход, если есть переполнение	$V = 1$
BGEQU	1E	Переход, если больше или равно (без знака)	$C = 0$
BCC	1E	Переход, если нет переноса	$C = 0$
BLSSU	1F	Переход, если меньше (без знака)	$C = 1$
BCS	1F	Переход, если есть перенос	$C = 1$

Сравнение или проверка на равенство осуществляется одинаково для чисел со знаком и без знака. Однако для того, чтобы избавить программиста от необходимости помнить, когда надо использовать различные переходы для чисел со знаком и без знака, предусмотрены мнемонические обозначения BEQLU и BNEQU, для которых КОП совпадают с кодами операций BEQL и BNEQ. Это позволяет программисту быть последовательным и использовать символ U каждый раз, когда он имеет дело с числами без знака.

Для анализа переполнения при обработке чисел без знака (наличия переноса из старшего разряда результата при выполнении арифметической операции) служат команды BCS и BCC, а для анализа переполнения при выполнении операций над числами со знаком используются команды BVS и BVC.

Любопытно отметить, что команда BLSSU идентична команде BCS (код операции ^X 1F), а команда BGEQU – команде BCC (код операции ^X 1E).

3.5. Команды организации циклов

Команды ACBB, ACBW, ACBL (табл. 3.5) включены в систему команд VAX для формирования циклов с проверкой условия их окончания после выполнения операций тела цикла. Эти команды эквивалентны оператору языков высокого уровня FOR...TO. Рассмотрим подробнее команду ACBB. Символическое имя кода операции команды ACBB содержит в себе начальные буквы названий выполняемых операций (Add – приращение, Compare – сравнение, Branch – передача управления, а также указание на размер сравниваемых данных, Byte – байт). Обобщенный формат команды ACBB имеет следующий вид:

ACBB предел, шаг, параметр цикла, адрес перехода

Отметим, что смещение в этих командах записывается в формате W.

П р и м е р. Организовать цикл для сложения чисел от 1 до 10 с использованием команды ACBB и без нее. В табл. 3.6 представлен мнемокод двух программ решения задачи. Первая не использует команду организации цикла, а вторая использует команду ACBB. Для хранения результата (суммы) используется регистр 2. Регистр 1 выполняет роль счетчика цикла.

Содержимое редактора памяти, в которой записаны обе программы (первая, начиная с адреса ^X 00000020, а вторая – с адреса ^X 00000040) представлено на рис. 3.17. Результат выполнения обеих программ одинаков и равен ^X 37 или 55 (10). Содержимое

Таблица 3.5

Команды организации циклов

Мнемокод	КОП	Название и формат	Описание
ACBB	9D	Сложение, сравнение и переход ACB [B, W, L] оп1, оп2, оп3, смещение	Содержимое оп2 и оп3 складывается, сумма запоминается в оп3. Затем содержимое оп3 и оп1 сравнивается, и если выполняется одно из двух условий: оп2 >= 0 и оп3 <= оп1, оп2 < 0 и оп3 >= оп1, то происходит переход по адресу, полученному путем прибавления смещения к содержимому PC
ACBW	3D		
ACBL	F1		

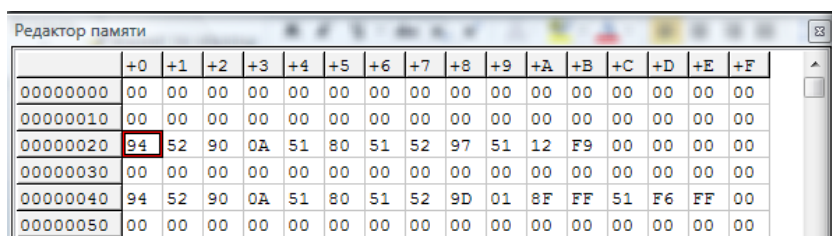
регистров после выполнения первой программы представлено на рис. 3.18.

Из примера можно сделать вывод, что вторая программа, использующая команду ACBB, позволяет организовать цикл с меньшим числом команд.

Таблица 3.6

Команды организации циклов

Первая программа	Вторая программа
CLRB R2 MOVB #10,R1 M1: ADDB2 R1,R2 DECB R1 BNEQ M1	CLRB R2 MOVB #10,R1 M1: ADDB2 R1,R2 ACBB #1,#-1,R1,M1



	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
00000000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000020	94	52	90	0A	51	80	51	52	97	51	12	F9	00	00	00	00
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000040	94	52	90	0A	51	80	51	52	9D	01	8F	FF	51	F6	FF	00
00000050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Рис. 3.17. Содержимое редактора памяти до выполнения программ

R2: 00000037	RA: 00000000
R3: 00000000	RB: 00000000
R4: 00000000	RC: 00000000
R5: 00000000	RD: 00000000
R6: 00000000	RE: 00000000
R7: 00000000	RF: 0000002d

Рис. 3.18. Содержимое регистров после выполнения первой программы

3.6. Команды специального назначения

Команды специального назначения приведены в табл. 3.7.

Команды BICPSW и BISPSW позволяют присваивать значения 0 или 1 содержимому отдельных разрядов регистра PSW (разряды 0–3) или их комбинации. Эти команды действуют аналогично командам BICW2 и BISW2 соответственно. Каждая команда имеет только один операнд – маску длиной 16 бит. Так как пользователь в эмулирующей программе может использовать только разряды 0–3 регистра PSW, то рекомендуется в указанных командах неиспользуемые разряды маски (4–15-й разряды) обнулять.

Таблица 3.7

Команды специального назначения

Мнемокод	КОП	Название и формат	Описание
HALT	00	HALT (останов)	Производится останов выполнения программы
NOP	01	NOP (нет операции)	Не производится никакой операции
BICPSW	B9	BICPSW оп1 (очистка разрядов слова состояния процессора)	Содержимое PSW логически умножается на инвертированное значение маски (содержимое оп1). Разряды 4–15-й маски должны быть равны 0
BISPSW	B8	BISPSW оп1 (установка разрядов слова состояния процессора)	Содержимое PSW логически складывается с маской (содержимое оп1). Разряды 4–15-й маски должны быть равны 0

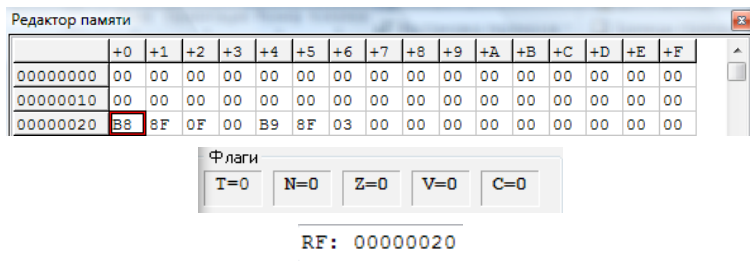


Рис. 3.19. Начальное содержимое редактора памяти и флагов

П р и м е р. Начальное содержимое редактора памяти и флагов приведено на рис. 3.19. Первая команда BISPSW работает с маской ^X 000F. Соответственно устанавливаются в 1 все четыре флага (N, Z, V, C) (рис. 3.20). Вторая команда BICPSW имеет маску ^X 0003 и сбрасывает флаги V и C в 0 (рис. 3.21).

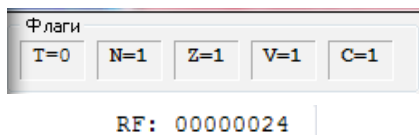


Рис. 3.20. Содержимое флагов после выполнения команды BISPSW

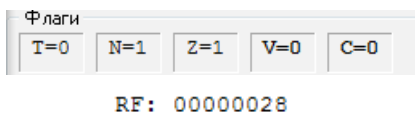


Рис. 3.21. Содержимое флагов после выполнения команды BICPSW

ЗАКЛЮЧЕНИЕ

В учебном пособии рассмотрены основные вопросы архитектуры ЭВМ: определение понятия архитектуры, принципы программного управления, структура классической ЭВМ фон Неймана, приведена классификация различных архитектур систем команд. Для более детального изучения можно рекомендовать [4–7].

Так как большинство персональных ЭВМ являются универсальными ЭВМ с CISC-архитектурой, то главной задачей пособия было изучение архитектуры универсальных ЭВМ на примере ЭВМ семейства VAX11, которая относится к классу CISC ЭВМ с регистровой архитектурой.

Данное пособие может быть использовано как при изучении дисциплин, связанных с архитектурой ЭВМ, так и при выполнении лабораторных работ с использованием эмулятора VAX11.

Библиографический список

1. *Кэпс. Ч. VAX : программирование на языке ассемблера и архитектура* / пер. И. В. Сперанской и др. М.: Радио и связь, 1991. 416 с.
2. *Майерс Г. Архитектура современных ЭВМ: в 2 кн.* / пер. В. К. Потоцкой. М.: Мир, 1985. 364 с. Кн. 1.
3. *Каршенбойм И. Стековые процессоры или новое – это хорошо забытое новое* // Компоненты и технологии. 2003. № 9. С. 128–133.
4. *Орлов С. А., Цилькер Б. Я. Организация ЭВМ и систем: учебник.* 2-е изд. СПб.: Питер, 2011. 686 с.
5. *Новожилов О.П. Архитектура ЭВМ и систем.* М.: Юрайт, 2012. 528 с.
6. *Патерсон Д., Хеннеси Дж. Архитектура компьютера и проектирование компьютерных систем.* СПб.: Питер, 2012. 784 с.
7. *Максимов Н. В., Партыка Т. Л., Попов И. И. Архитектура ЭВМ и вычислительных систем: учебник.* 5-е изд., перераб. и доп. М.: Форум, Инфра-М, 2013. 512 с.